Cricket Player Performance Prediction

Submitted by:

Soumya S Betgeri

ABSTRACT

Cricket generates vast amounts of structured and semi-structured data, yet player selection and strategy decisions are often influenced by intuition and traditional averages. This project presents a machine learning-based system designed to predict individual player performance in the Indian Premier League (IPL). The system leverages historical ball-by-ball data to forecast runs scored by batsmen and wickets taken by bowlers in upcoming matches.

The methodology involves data preprocessing, exploratory data analysis, feature engineering, regression model development, and model deployment through an interactive Streamlit dashboard. Multiple machine learning models including Random Forest, LightGBM, and XGBoost were trained and compared against a baseline rolling average model. Model evaluation was conducted using MAE, RMSE, and R^2 metrics. SHAP was used for interpretability to understand feature influence.

The final system provides accurate and explainable predictions and offers an interactive interface for analysts and users. This project demonstrates how machine learning can enhance sports analytics and data-driven decision-making in cricket.

CHAPTER 1

INTRODUCTION

1.1 Background

The Indian Premier League (IPL) is one of the most data-rich sporting leagues globally. Each match generates detailed ball-by-ball data including runs, wickets, player contributions, and contextual match conditions. Despite the availability of such granular data, performance forecasting often relies on simple averages or expert judgment.

With advancements in machine learning, it is possible to model nonlinear patterns and contextual relationships to generate more reliable predictions.

1.2 Motivation

Traditional averages fail to capture:

- Recent form trends
- Venue-specific performance
- Opponent-specific matchups
- Performance consistency

This project aims to bridge this gap using data-driven predictive modelling.

1.3 Problem Statement

To design and implement a machine learning system that predicts:

- Runs scored by a batsman in an upcoming IPL match
- Wickets taken by a bowler in an upcoming IPL match

Using historical IPL ball-by-ball and match data.

1.4 Objectives

- Collect and preprocess historical cricket data
- Identify key performance indicators affecting player output
- Develop machine learning models for prediction
- Evaluate prediction accuracy using statistical metrics
- Build an interactive dashboard for visualization
- Provide explainability using feature importance

CHAPTER 2

PROJECT SCOPE

2.1 In-Scope

- IPL T20 format (2008–latest season)
- Ball-by-ball historical data
- Regression-based performance prediction
- Model interpretability using SHAP
- Interactive dashboard deployment

CHAPTER 3

TECHNICAL IMPLEMENTATION

3.1 Development Environment

The project was implemented using the following environment:

- Programming Language: Python 3
- IDE: VS Code
- Libraries Used:
    - pandas, numpy – Data handling
    - matplotlib, seaborn – Visualization
    - scikit-learn – ML models & preprocessing
    - xgboost, lightgbm – Gradient boosting models
    - shap – Model explainability
    - joblib – Model serialization
    - streamlit – Dashboard deployment

CHAPTER 4

DATASET DESCRIPTION

Source : Kaggle

IPL ball-by-ball

Dataset	Rows	Columns
Matches.csv	1,095	20
Deleveries.csv	260,920	17

The dataset used for this project was obtained from the Kaggle IPL archives and consists of two main files: matches.csv and deliveries.csv. The matches.csv file contains match-level information such as season, date, venue, participating teams, toss details, and match results. This file provides contextual information that helps in understanding external factors affecting player performance, such as venue conditions and opposition strength. The deliveries.csv file contains detailed ball-by-ball data, including batsman, bowler, runs scored, extras, dismissals, and over information. This granular structure enables precise calculation of player performance metrics.

Both datasets were merged using the common match_id column to combine match context with ball-level performance data. After merging, the dataset was aggregated to the player-match level to calculate total runs for batsmen and total wickets for bowlers per match. Missing values were handled appropriately, column names were standardized, and date formats were converted to datetime for time-based analysis. This cleaned and structured dataset formed the foundation for feature engineering, model training, and performance prediction

CHAPTER 5

METHODOLOGY

The project follows a structured machine learning pipeline.

5.1 Data Collection

Historical IPL data was collected from Kaggle's IPL dataset repository. The dataset consists of two primary files:

- **matches.csv** – Contains match-level details such as venue, teams, toss result, and match winner.
- **deliveries.csv** – Contains ball-by-ball information including batsman runs, bowler details, dismissals, and extras.

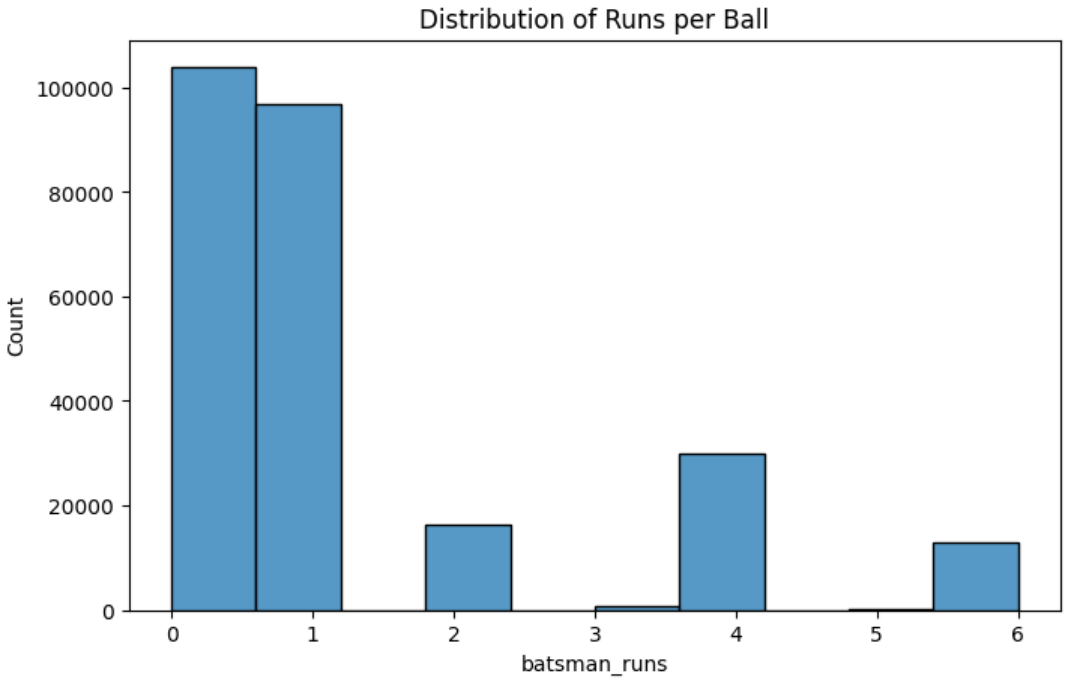
5.2 Data Preprocessing

- Missing value handling
- Column normalization
- Date formatting
- Aggregation to player-match level
- Feature scaling using StandardScaler

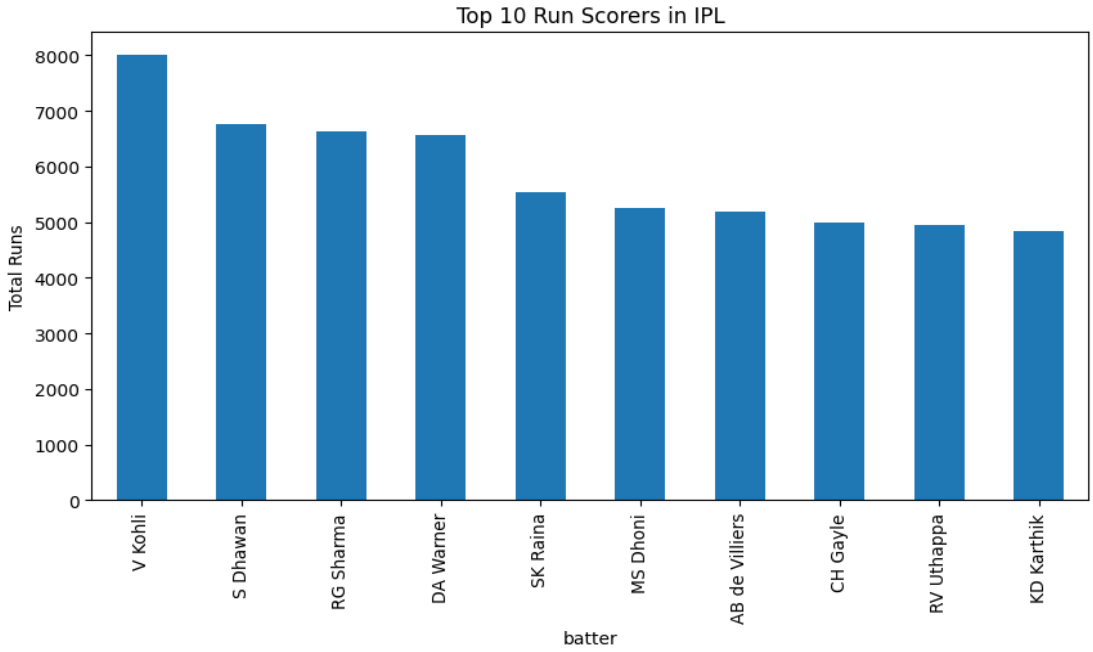
5.3 Exploratory Data Analysis

EDA was conducted to understand performance patterns and data distributions before modelling.

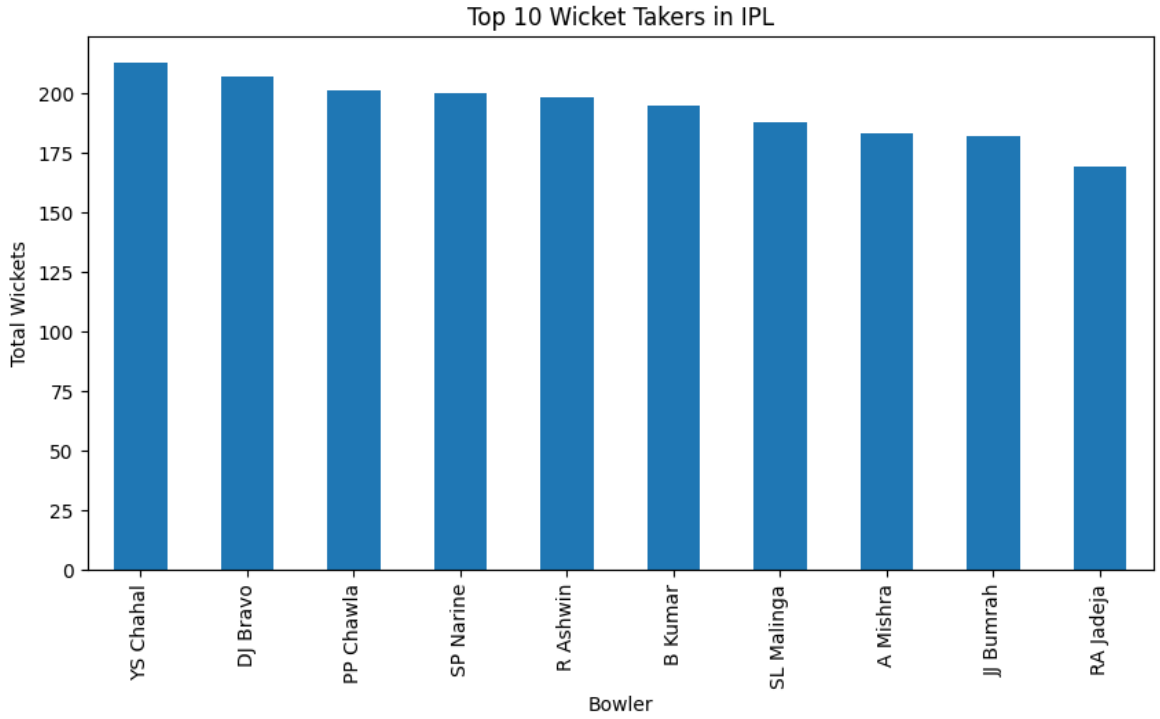
5.3.1 Runs distribution



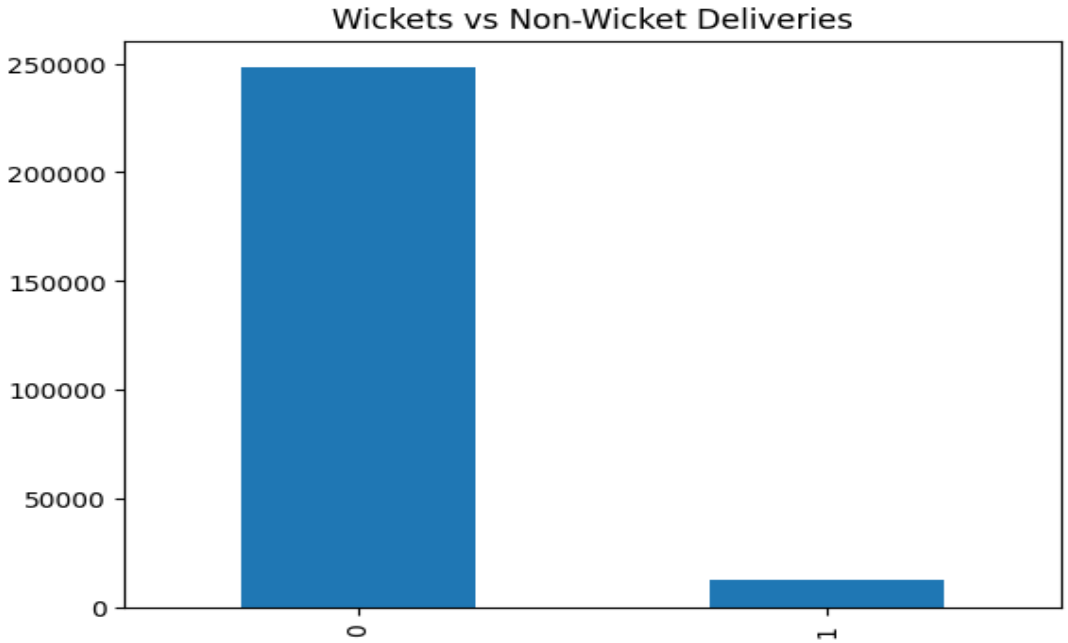
5.3.2 Top run scorer in IPL



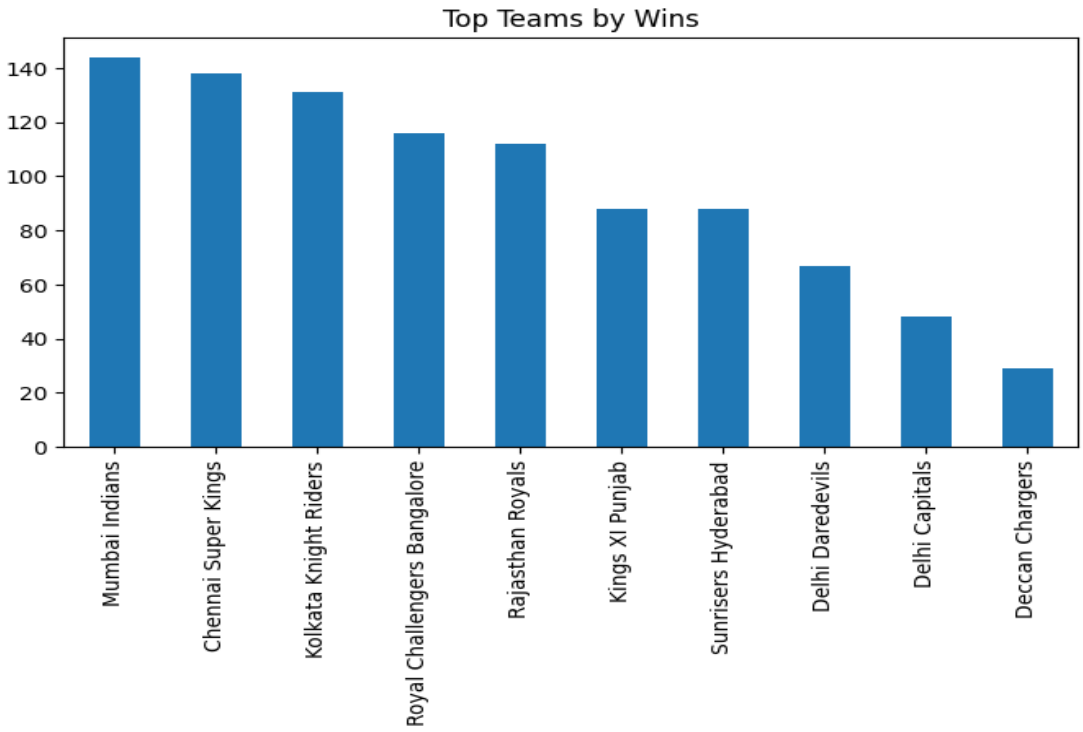
5.3.3 Top 10 wicket taker in IPL



5.3.4 Wicket Distribution



5.3.5 Team Performance



5.4 Feature Engineering

Feature engineering is the most critical stage of this project. Instead of relying on raw statistics, advanced cricket-specific features were engineered.

Engineered features include:

- Career average
- Recent form (rolling averages)
- Venue average
- Opponent-specific average
- Standard deviation (consistency)
- Match count

5.5 Train-Test Split

Since cricket performance is time-dependent, a time-series aware split was used instead of random splitting.

- Older seasons were used for training.
- Newer matches were reserved for testing.

This approach prevents data leakage, ensuring that the model predicts future performance based only on past information.

Final Split Example:

- Training Set: ~80%
- Testing Set: ~20%

5.6 Model Development

Multiple models were implemented and compared.

5.6.1 Baseline Model – Rolling Average

A simple 10-match rolling average was used as a benchmark. This provides a performance reference to evaluate ML improvements.

5.6.2 Random Forest

An ensemble tree-based model that reduces variance and handles non-linearity effectively.

5.6.3 XGBoost

Gradient boosting framework optimized for performance and regularization.

5.6.4 LightGBM

Faster gradient boosting model suitable for large datasets. Hyperparameter tuning was performed using GridSearchCV to optimize performance.

5.7 Model Evaluation

Model performance was evaluated using regression metrics:

• MAE (Mean Absolute Error)

Measures average prediction error magnitude. Lower MAE indicates better accuracy.

• RMSE (Root Mean Squared Error)

Penalizes larger errors more than MAE. Useful for detecting large prediction deviations.

• R^2 Score

Indicates how well the model explains variance in the data. Closer to 1 indicates better model performance.

5.7.1 Model Results

Model Performance Comparison

Model	MAE	RMSE	R^2 Score
Baseline (Rolling Avg)	16.506518	22.259986	-0.00469
Random Forest	16.2294	21.265204	0.083101
XGBoost (Best Model)	15.673821	20.810804	0.121867
LightGBM	15.674209	20.929625	0.111811

The table presents a comparison of different models used to predict player performance based on three evaluation metrics: MAE, RMSE, and R^2 score. The baseline model using a rolling average shows the weakest performance, with the highest error values and a negative R^2 score (-0.00469), indicating that it fails to explain the variance in the data effectively. The Random Forest model improves performance slightly by reducing error values and achieving a positive R^2 score (0.0831), meaning it captures some patterns in the data. LightGBM further reduces prediction errors and increases the R^2 score to 0.1118. However, XGBoost performs the best among all models, achieving the lowest MAE (15.67), lowest RMSE (20.81), and the highest R^2 score (0.1218), indicating that it provides the most accurate and reliable predictions. Therefore, XGBoost was selected as the final model for deployment in the dashboard.

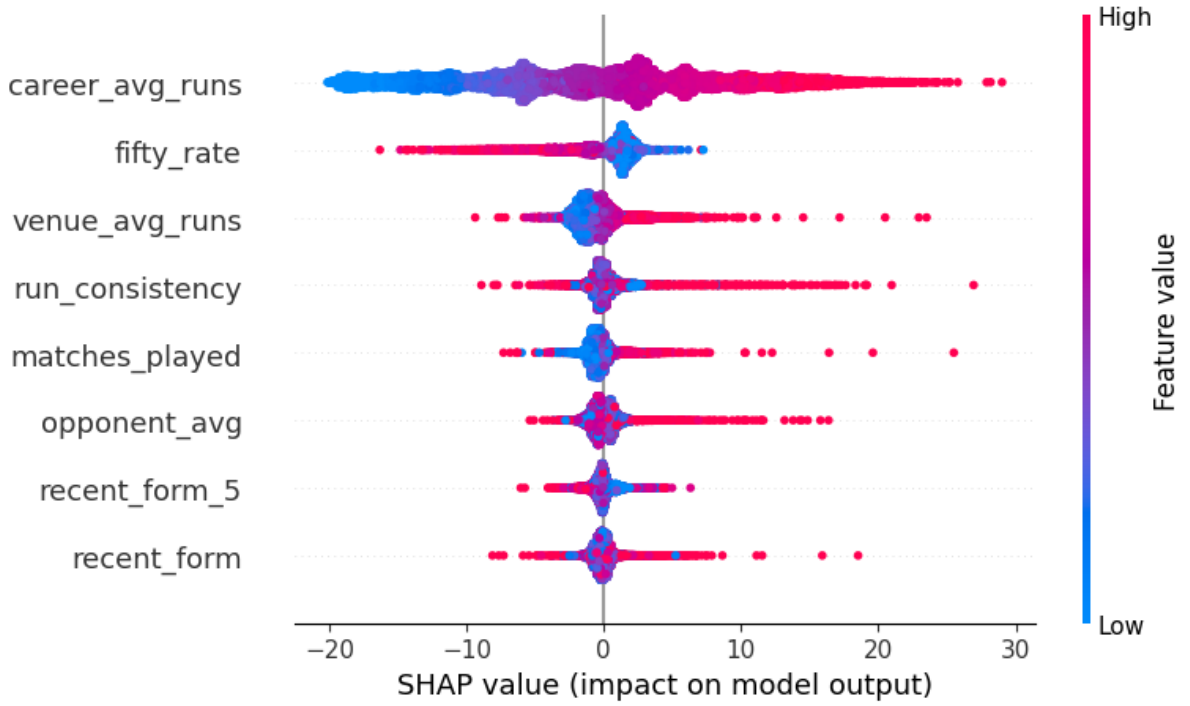
5.8 Deployment

Final XGBoost model integrated into Streamlit dashboard.

CHAPTER 6:

MODEL INTERPRETATION

SHAP analysis revealed:



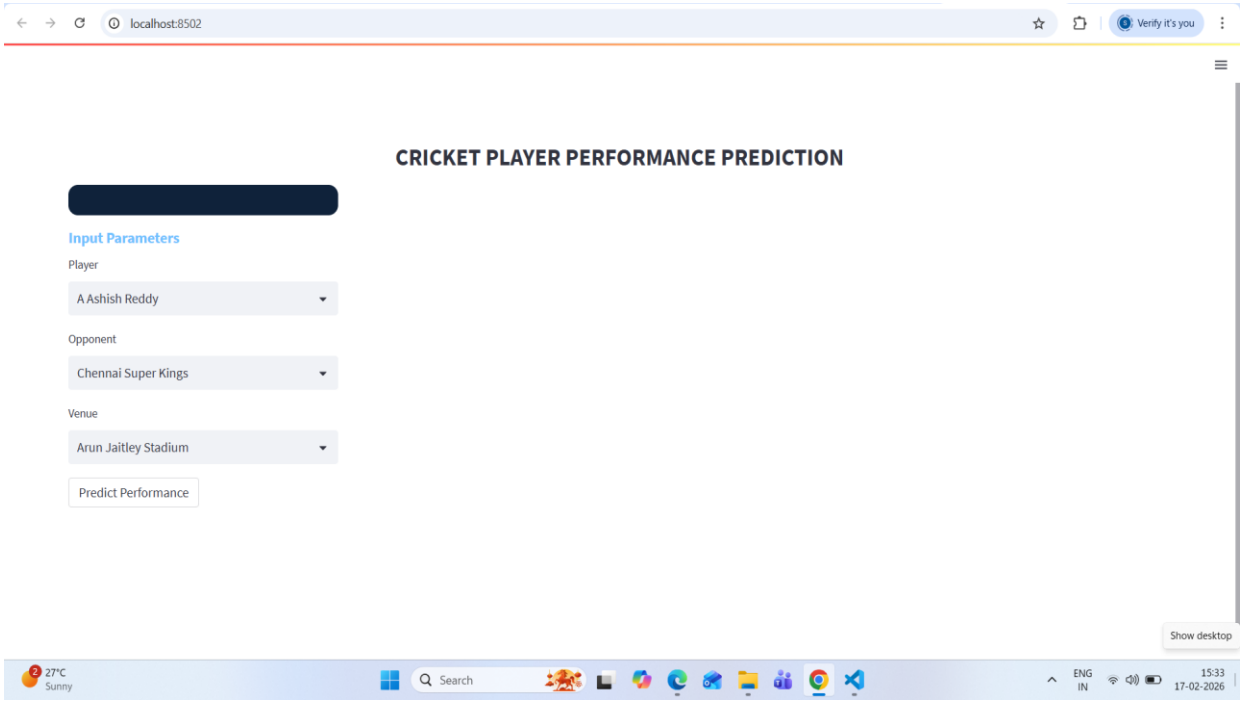
This image is a SHAP summary plot, which shows how different features influence the model's predictions for player runs. Each row represents a feature, ordered from most important (top) to least important (bottom). The x-axis shows the SHAP value, which indicates how much that feature increases (positive value) or decreases (negative value) the predicted runs. The color represents the feature value, where red indicates high values and blue indicates low values.

From the plot, career_avg_runs is the most influential feature, meaning a player's overall career average has the strongest impact on predicted performance. Higher career averages (red points) generally push predictions upward. Features like fifty_rate and venue_avg_runs also significantly affect predictions, showing that recent scoring ability and performance at specific venues matter. Variables such as run_consistency, matches_played, and opponent_avg contribute moderately, while recent_form and recent_form_5 still influence predictions but to a lesser extent. Overall, the plot confirms that long-term performance metrics and venue-based statistics are the key drivers in the model's decision-making process.

CHAPTER 7:

DASHBOARD IMPLEMENTATION

7.1 User Input Interface



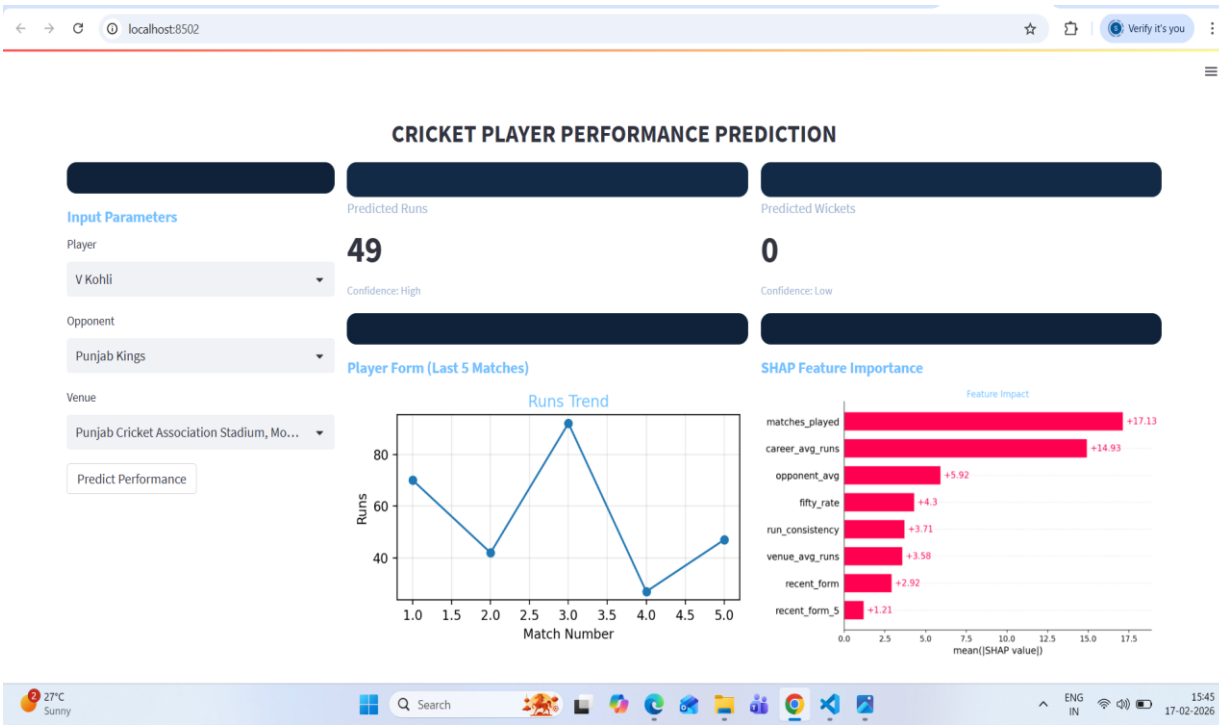
The dashboard was developed using the Streamlit framework to provide an interactive and user-friendly interface for performance prediction. The input interface serves as the primary interaction layer between the user and the predictive model. It enables users to specify match-related parameters required for generating performance predictions.

The input interface consists of three primary selection components: Player, Opponent, and Venue. The Player dropdown allows users to select a specific batsman or bowler whose performance is to be predicted. Upon selection, the system dynamically retrieves the player's historical statistics from the processed dataset.

The Opponent dropdown enables selection of the opposing team, which is used to compute opponent-specific performance metrics. Similarly, the Venue dropdown allows users to select the match stadium, facilitating the calculation of venue-based averages.

Once all input parameters are selected, the user initiates the prediction process by clicking the “Predict Performance” button. This action triggers the backend processing pipeline. The system extracts historical data corresponding to the selected player and computes engineered features such as career average, recent form (rolling average of last five matches), venue average, opponent average, consistency metrics, and total matches played. These features are then passed through a pre-trained preprocessing pipeline (saved as feature_pipeline.pkl) to ensure consistent scaling and transformation. The processed feature vector is subsequently fed into the trained XGBoost model, which generates the final prediction.

7.2 Output Interface and Analytical Visualization



The output interface presents prediction results along with interpretability insights in a structured and visually intuitive format. The primary output component is the Predicted Runs (for batsmen) or Predicted Wickets (for bowlers), displayed prominently for immediate interpretation. In addition, a confidence indicator is provided to communicate the model's certainty level regarding the prediction.

To enhance interpretability, the dashboard includes a Player Form visualization representing performance in the last five matches. This line graph enables users to assess recent performance trends and consistency. Such visualization supports contextual understanding of the prediction.

Furthermore, SHAP (SHapley Additive exPlanations) feature importance plots are integrated into the dashboard. These plots display the contribution of each feature to the final prediction, thereby increasing model transparency. Features such as career average, matches played, opponent average, and venue average are shown with their respective impact magnitudes. This explainability component ensures that the model operates as an interpretable decision-support system rather than a black-box predictor.

7.3 System Workflow and Architecture

The dashboard follows a structured end-to-end workflow integrating data processing, machine learning modeling, and real-time visualization. The operational flow is as follows:

1. The user provides input parameters through the Streamlit interface.
2. The backend retrieves relevant historical data from the processed dataset.
3. The feature engineering module computes contextual performance metrics.
4. The pre-saved preprocessing pipeline transforms the feature set.
5. The trained XGBoost model generates the prediction.
6. SHAP analysis computes feature-level contribution scores.
7. The dashboard displays predictions and visual insights to the user.

The model artifacts are stored using the Joblib library, ensuring efficient loading and deployment. This architecture enables seamless integration between preprocessing, model inference, and visualization components.

CHAPTER 8:

ADVANTAGES OF THE DASHBOARD

The developed cricket player performance prediction dashboard offers several practical and technical advantages that enhance analytical decision-making and usability.

1. **Real-Time Prediction:** Provides instant player performance predictions based on selected match conditions.
2. **User-Friendly Interface:** Simple and interactive design allows easy usage without technical knowledge.
3. **Data-Driven Decisions:** Supports informed decision-making using historical data analysis.
4. **Recent Form Visualization:** Displays last few match performances for better context.
5. **Explainable AI (SHAP):** Shows which features influence the prediction.
6. **Reduced Manual Analysis:** Eliminates the need for manual statistical calculations.
7. **Interactive Visualization:** Presents results in clear graphical format.
8. **Scalable and Deployable:** Can be deployed online and extended with additional features.

CHAPTER 9:

FUTURE SCOPE

The future scope of this project includes enhancing the prediction system by integrating additional contextual data such as weather conditions, pitch reports, and player fitness information to improve accuracy and reliability. The model can be extended to other cricket formats such as ODI and Test matches, enabling broader applicability. Advanced deep learning techniques like LSTM and RNN can be implemented to better capture time-series performance trends. The system can also be upgraded to support live match predictions with real-time data updates. Furthermore, features such as fantasy team recommendations, player comparison analysis, risk scoring indicators, and win probability prediction can be incorporated. Deploying the application on cloud platforms and developing a mobile-friendly interface would improve accessibility and scalability. Automated seasonal retraining can also be implemented to handle concept drift and maintain long-term model performance.

CONCLUSION

This project successfully demonstrates the application of machine learning techniques in sports analytics. The XGBoost-based prediction system, combined with SHAP interpretability and a Streamlit dashboard, provides accurate and explainable cricket performance forecasts.

The system highlights how contextual and temporal features significantly enhance predictive accuracy over traditional averages. The project serves as a strong foundation for advanced cricket analytics applications.